CONTRACT V1

Mục tiêu: năm thành viên phát triển bốn thuật toán độc lập, tích hợp vào game với ít chỉnh sửa nhất.

Nguyên tắc: giữ các lớp `Block`, `Board`, `State`, `Problem`, bộ đọc màn và GUI hiện có. Chỉ bổ sung giao diện kết quả, giới hạn chạy và lớp chuyển đổi cho GUI. Không bắt buộc đổi cấu trúc trạng thái sang mô hình mới.

1. Cấu trúc mã nguồn

Giữ nguyên cấu trúc repo, bổ sung:

```
gameModel/
    block.py              # Dùng lại
    board.py              # Dùng lại
    state.py              # Dùng lại, sửa khi cần bảo đảm đúng
    problem.py            # Giao diện chung hiện có
    gameController.py     # Dùng lại

search/
    contracts.py          # Mới: SearchLimits, SearchResult
    bfs.py                # Người 1
    dfs.py                # Người 2
    ucs.py                # Người 3
    astar.py              # Người 4
    heuristics.py         # Người 5
    registry.py           # Mới: chọn thuật toán
    solver.py             # Giữ tạm nếu GUI hiện đang import

runner.py                 # Mới: gọi solver và đo hiệu năng
view.py                   # Chỉnh chỗ gọi và nhận lời giải
```

Các thuật toán cũ được giữ làm tham khảo, nhưng phải phân biệt rõ với mã nhóm tự viết.

2. Giao diện bài toán

Bốn thuật toán sử dụng giao diện đã có:

problem.initial
problem.actions(state)
problem.result(state, action)
problem.goal_test(state)
problem.step_cost(state, action, next_state)

Thành phần	Quy định
`initial`	Trạng thái bắt đầu tìm kiếm
`actions(state)`	Trả iterable các hành động có thể thử, theo thứ tự xác định
`result(state, action)`	Trả trạng thái mới hoặc `None` nếu không hợp lệ
`goal_test(state)`	Đúng khi khối đầy đủ đứng trên đích
`step_cost(...)`	Trả chi phí dương, hữu hạn của chuyển trạng thái hợp lệ

Không đổi tên các hàm trên. Không bắt buộc `actions()` phải trả `tuple`; có thể giữ kiểu danh sách hiện tại.

Thuật toán không tự áp dụng luật, không truy cập GUI và không sửa `problem.initial`.

3. Hành động

Dùng trực tiếp **`Direction`** **trong** **`gameModel.block`**, không tạo enum `Action` trùng chức năng.

from gameModel.block import Direction

Quy định:

- Giữ nguyên các tên và giá trị hướng đang có trong repo.
- `Problem.actions()` cung cấp thứ tự duyệt xác định; tất cả thuật toán lấy hành động từ hàm này.
- Không tự viết cứng danh sách bốn hướng bên trong thuật toán.
- Khi bổ sung tách khối, mở rộng enum chung bằng `SWITCH` và cập nhật engine, GUI tương ứng.
- `SWITCH` chỉ đổi khối điều khiển, không di chuyển hoặc tự kích hoạt lại công tắc.
- Không gộp `SWITCH` và một hướng thành một hành động.

GUI nhận lại đúng các đối tượng `Direction` mà engine sử dụng, không cần đổi sang chuỗi trong quá trình chạy.

4. Trạng thái

Tiếp tục sử dụng lớp hiện tại:

from gameModel.state import State

initial_state = State(board, block)

Không bắt buộc thay `board`, `block` bằng các trường mới. Tuy nhiên, phải đáp ứng các yêu cầu sau:

Yêu cầu	Ý nghĩa
So sánh đầy đủ	Cùng vị trí nhưng khác cấu hình cầu phải được phân biệt
Băm nhất quán	Nếu `a == b` thì `hash(a) == hash(b)`
Sao chép độc lập	Thay đổi trạng thái con không ảnh hưởng cha hoặc nhánh khác
Không sửa khi đã lưu	Trạng thái trong `set`/`dict` không được tiếp tục thay đổi
Không có đồ họa	Không đưa đối tượng Ursina vào trạng thái tìm kiếm

Khi thêm luật nâng cao, cập nhật sao chép, so sánh và băm để bao gồm:

- Vị trí, tư thế khối.
- Trạng thái động của cầu.
- Vị trí hai khối con và khối đang điều khiển.
- Thông tin động khác nếu ảnh hưởng tới bước tiếp theo.

Người viết thuật toán không phụ thuộc cấu trúc bên trong `State`. Người viết heuristic có thể đọc `state.block`, `state.board`, nhưng phải phối hợp khi cấu trúc thay đổi.

Node cha, chi phí tích lũy và đường đi do thuật toán quản lý riêng.

5. Cam kết của engine

`Problem.result()` phải:

1. Không sửa trạng thái đầu vào.
2. Không thay đổi game đang hiển thị.
3. Không chạy hoạt ảnh hoặc chờ thời gian.
4. Trả kết quả xác định với cùng trạng thái và hành động.
5. Xử lý toàn bộ tương tác của hành động trước khi trả về.
6. Trả `None` nếu trạng thái cuối không hợp lệ.

Với CS-AI, tiếp tục dùng `State.copy()` và các hàm luật hiện có; chỉ sửa những phần không đáp ứng các cam kết này.

Phần chơi/phát lại trong `GameController` phải cho kết quả tương đương `Problem.result()`. Có thể dùng lại các hàm luật chung từng bước, **không bắt buộc tái cấu trúc toàn bộ trước khi viết thuật toán**.

6. Chữ ký chung của solver

Mỗi tệp `bfs.py`, `dfs.py`, `ucs.py`, `astar.py` cung cấp:

```
def solve(
    problem,
    limits,
    heuristic=None,
) -> SearchResult:
    ...
```

- BFS, DFS, UCS bỏ qua `heuristic`.
- A* dùng `heuristic(problem, state)`; nếu `None` thì dùng `0`.
- Solver không mở cửa sổ, không điều khiển GUI.
- Solver không tự bật/tắt `tracemalloc`.
- Mỗi người được chọn cấu trúc node nội bộ.
- Không bắt buộc dùng `Node` hiện có nếu không phù hợp.

Vòng gọi engine chung:

```
for action in problem.actions(state):
    child_state = problem.result(state, action)

    if child_state is None:
        continue
```

```
    step = problem.step_cost(state, action, child_state)
    # Xử lý frontier, trạng thái lặp và truy vết
```

7. Đầu vào và đầu ra chung

Tạo `search/contracts.py`:

```
from dataclasses import dataclass
from typing import Literal

from gameModel.block import Direction


SearchStatus = Literal[
    "solved",
    "unsolvable",
    "timeout",
    "node_limit",
]


@dataclass(frozen=True)
class SearchLimits:
    timeout_seconds: float = 60.0
    max_expanded_nodes: int = 1_000_000


@dataclass(frozen=True)
class SearchResult:
    status: SearchStatus
    actions: tuple[Direction, ...]
    total_cost: float | None
    expanded_nodes: int
    generated_nodes: int
```

Quy tắc trả kết quả

Trường hợp	`status`	`actions`	`total_cost`

Có lời giải	`solved`	Chuỗi hành động	Tổng chi phí
Ban đầu đã thắng	`solved`	`()`	`0.0`
Duyệt hết không có lời giải	`unsolvable`	`()`	`None`
Hết thời gian	`timeout`	`()`	`None`
Đạt giới hạn node	`node_limit`	`()`	`None`

Solver tự truy vết trước khi trả về. **Không trả node nội bộ cho GUI.**

Exception được runner báo là lỗi chương trình/dữ liệu, không chuyển thành `unsolvable`.

8. Quy tắc tìm kiếm và giới hạn

Nội dung	Quy định
Kiểm tra đích	Khi lấy node ra khỏi frontier để xét
BFS	Đánh dấu đã phát hiện khi đưa vào queue
DFS	Chống lặp; nếu dùng stack, đẩy successor ngược thứ tự để xét theo thứ tự chung
UCS/A*	Quản lý `best_g`, cập nhật đường rẻ hơn, bỏ mục heap lỗi thời
A*	Cho phép mở lại trạng thái nếu tìm được đường rẻ hơn
Phá hòa heap	Dùng bộ đếm tăng dần, không so sánh trực tiếp `State`

Giới hạn:

- `timeout_seconds > 0`.
- `max_expanded_nodes >= 0`.
- Solver kiểm tra thời gian bằng `time.perf_counter()`.
- Kiểm tra định kỳ ở vòng lặp và giữa các lần sinh trạng thái.
- Không được trả `unsolvable` chỉ vì hết thời gian.

Thứ tự xử lý:

1. Bỏ mục lỗi thời hoặc node đã bị trội.
2. Kiểm tra đích.
3. Kiểm tra timeout và giới hạn node.
4. Mở rộng node.

9. Chi phí và heuristic

Giữ giao diện `Problem.step_cost()` và cấu hình chi phí hiện có để bắt đầu. Không để từng người tự đặt một hàm chi phí khác nhau.

- UCS và A* dùng cùng hàm chi phí.
- BFS/DFS cũng tính tổng chi phí của lời giải để báo cáo.
- Có cấu hình chi phí bằng 1 để kiểm thử.
- Trước thực nghiệm chính thức, rà soát hàm chi phí không đồng nhất và giải thích trong báo cáo.
- Khi thêm `SWITCH`, phải xác định chi phí dương cho hành động này.

Heuristic:

```
def heuristic(problem, state) -> float:
    ...
```

- Hữu hạn, không âm, bằng 0 tại đích.
- Ban đầu dùng `h=0` để kiểm tra A* với UCS.
- Heuristic hiện có của repo chỉ là tham khảo, chưa mặc định đúng admissible.
- Thêm teleport hoặc đổi chi phí phải kiểm chứng lại heuristic.

10. Số liệu thực nghiệm

Chỉ số	Định nghĩa
`expanded_nodes`	Số lần thực sự sinh successor; không tính node đích và mục lỗi thời
`generated_nodes`	Số successor hợp lệ trước lọc trùng; không tính trạng thái đầu
Độ dài lời giải	`len(result.actions)`, gồm `SWITCH`
Số lần lăn	Chỉ đếm hành động di chuyển

Tổng chi phí	Tổng `step_cost()` của lời giải
Thời gian	Toàn bộ lời gọi `solve()`, gồm truy vết; không gồm đọc màn, kiểm chứng hoặc hoạt ảnh
Bộ nhớ	Đỉnh bộ nhớ Python do `tracemalloc` ghi nhận, đơn vị byte; không phải RSS

Runner đo thời gian và bộ nhớ chung, luôn dừng công cụ đo khi hoàn tất hoặc gặp lỗi. Không chạy nhiều phép đo `tracemalloc` đồng thời trong cùng tiến trình.

11. Tích hợp GUI với ít thay đổi

Giữ GUI hiện tại, chỉ chỉnh điểm gọi solver và nhận kết quả:

```
result = run_search(problem, algorithm, limits, heuristic)

if result.search.status == "solved":
    self.autoSolution = list(result.search.actions)
    self.autoIndex = 0
    # Gọi cơ chế playSolution() hiện có
```

`run_search()` trả một đối tượng chuẩn:

```
@dataclass(frozen=True)
class RunResult:
    search: SearchResult
    search_time_seconds: float
    peak_memory_bytes: int
```

Không cần thay lại toàn bộ menu hoặc cơ chế phát lời giải. Lớp `registry.py` ánh xạ lựa chọn trên GUI tới các hàm `solve()` tương ứng.

GUI phải phát từ đúng trạng thái ban đầu solver đã nhận và chờ một bước hoàn tất trước bước tiếp theo.

12. Phân công

Người	Phần chính

1	`search/bfs.py`
2	`search/dfs.py`
3	`search/ucs.py`
4	`search/astar.py`
5	`search/heuristics.py`, kiểm chứng A*

Chỉ định một người duyệt sửa đổi `State`, `Problem`, luật nâng cao và các tệp giao diện chung. AI có thể hỗ trợ sinh mã, nhưng thay đổi phải được review và kiểm thử.

13. Điều kiện bắt đầu và nghiệm thu

Trước khi chia nhánh:

- Repo chạy được một màn cơ bản.
- Giữ được giao diện `Problem` hiện có.
- Có `SearchLimits`, `SearchResult` và một ví dụ gọi solver.
- Kiểm tra `result()` không sửa cha hoặc nhánh khác.
- Kiểm tra trạng thái bằng nhau có cùng hash.

Khi tích hợp:

- Lời giải được phát lại hợp lệ và kết thúc ở đích.
- Tổng chi phí trả về khớp với chi phí phát lại.
- BFS và UCS cùng độ dài tối ưu khi chi phí bằng 1.
- A* với `h=0` cùng chi phí tối ưu với UCS.
- Màn không có lời giải, timeout và node limit được phân biệt.
- Khi thêm luật nâng cao, chạy lại các kiểm tra trên.

Phần giữ nguyên: mô hình game, tải màn, GUI và tên giao diện `Problem`.
Phần bổ sung: kết quả/giới hạn chuẩn, runner, bộ chọn thuật toán và bốn solver tự viết.
Phần sửa theo nhu cầu: tính độc lập của trạng thái, lỗi luật và các cơ chế nâng cao còn thiếu.